

Guia Completo: SQL DML - Consultas Complexas, Grupos e Junções

Introdução às Consultas SQL Complexas

Consultas SQL complexas são operações de banco de dados que transcendem a recuperação simples de informações de uma única tabela. Elas envolvem a combinação de dados de múltiplas fontes, a aplicação de condições elaboradas e a utilização de subconsultas para extrair insights específicos e detalhados de um banco de dados relacional. A necessidade de tais consultas surge frequentemente em cenários que exigem relatórios avançados, análises de dados aprofundadas e a extração de informações muito específicas de grandes volumes de dados ¹.

A complexidade de uma consulta SQL é determinada pelo uso de diversas cláusulas, como `SELECT`, `FROM`, `JOIN`, `WHERE`, `GROUP BY`, `HAVING` e `ORDER BY`. Um bom entendimento da estrutura do banco de dados, das relações entre as tabelas e da própria linguagem SQL é fundamental para construir e otimizar essas consultas ¹.

Cenários Comuns para Consultas Complexas:

- **Junção de Tabelas:** Combinação de dados de várias tabelas usando cláusulas `JOIN` para criar um conjunto de resultados mais abrangente.
- **Subconsultas:** Inclusão de consultas aninhadas dentro da consulta principal para buscar informações em múltiplas etapas ou aplicar condições específicas.
- **Agregação de Dados:** Utilização de funções de agregação (`SUM`, `AVG`, `COUNT`, `MAX`, `MIN`) para calcular valores resumidos com base em grupos de dados.
- **Filtragem Avançada:** Aplicação de condições complexas com operadores lógicos (`AND`, `OR`) para restringir os resultados.
- **Ordenação e Paginação:** Classificação dos resultados em uma ordem específica e aplicação de limites para controle de paginação.
- **Tratamento de Datas e Horas:** Manipulação de datas e horas para cálculos, filtragem por intervalos ou conversão de formatos.
- **Manipulação de Cadeias de Caracteres:** Uso de funções de texto para formatar ou extrair informações de campos de texto.
- **Expressões Condicionais:** Aplicação de expressões `CASE` para criar lógica condicional dentro da consulta ¹.

Operadores Baseados em Strings: LIKE e NOT LIKE

Os operadores `LIKE` e `NOT LIKE` são empregados na cláusula `WHERE` para realizar buscas por padrões específicos em colunas de texto (tipos `CHAR` ou `VARCHAR`). Eles permitem uma filtragem flexível, utilizando caracteres curinga para representar partes desconhecidas do padrão de busca ¹.

Caracteres Curinga:

- `%` (**Porcentagem**): Substitui zero, um ou múltiplos caracteres. Por exemplo, `LIKE 'LÁPIS%'` encontraria 'Lápis preto', 'Lápis Cera', 'Lápis Borracha' ¹.
- `_` (**Sublinhado**): Substitui um único caractere. Por exemplo, `LIKE 'Broca N_'` encontraria 'Broca N1', 'Broca N9', 'Broca N3' ¹.

Exemplos Avançados (Variações podem existir entre SGBDs):

Padrão SQL	Descrição	Exemplo de Correspondência
<code>'%ão'</code>	Qualquer nome que termine em 'ão'.	'Coração', 'Ação'
<code>'[CM]%'</code>	Qualquer nome que comece com 'C' ou 'M'.	'Carlos', 'Maria'
<code>'[C-X]%'</code>	Qualquer nome que comece com uma letra de 'C' a 'X'.	'Daniel', 'Felipe'
<code>'M[^o]%'</code>	Qualquer nome que comece com 'M' e a segunda letra não seja 'o'.	'Marta', 'Miguel'

Tratamento de Valores Desconhecidos: IS NULL e IS NOT NULL

Em SQL, `NULL` representa a ausência de um valor ou um valor desconhecido. É crucial entender que `NULL` não é o mesmo que zero ou uma string vazia. A comparação direta com `NULL` usando operadores como `=` ou `!=` sempre resultará em `FALSE` (ou `UNKNOWN`, dependendo do contexto), pois não se pode comparar um valor desconhecido com outro. Para verificar a presença ou ausência de valores nulos, utilizam-se os operadores `IS NULL` e `IS NOT NULL` ¹.

SQL

```
SELECT <campos>
FROM <tabela>
WHERE <Nome da coluna> IS NULL;

SELECT <campos>
FROM <tabela>
WHERE <Nome da coluna> IS NOT NULL;
```

Agregação e Agrupamento de Dados

A agregação de dados é o processo de resumir informações de um conjunto de linhas em um único valor. As funções de agregação são essenciais para análises estatísticas e para obter uma visão consolidada dos dados. A cláusula `GROUP BY` é utilizada em conjunto com funções de agregação para dividir o conjunto de resultados em grupos, permitindo que as funções de agregação operem em cada grupo individualmente ¹.

Funções de Agregação Comuns:

- `MIN()` : Retorna o menor valor em um conjunto.
- `MAX()` : Retorna o maior valor em um conjunto.
- `AVG()` : Calcula a média dos valores em um conjunto.
- `SUM()` : Calcula a soma total dos valores em um conjunto.
- `COUNT()` : Conta o número de linhas em um conjunto ¹.

Cláusula GROUP BY:

A cláusula `GROUP BY` agrupa linhas que têm os mesmos valores em colunas especificadas em um conjunto de linhas de resumo. Para cada grupo, você pode aplicar uma função de agregação. É uma regra fundamental que todas as colunas listadas na cláusula `SELECT` que não são funções de agregação devem ser incluídas na cláusula `GROUP BY` ¹.

Exemplo:

SQL

```
SELECT id_comissao, SUM(valor) AS ValorTotalComissao
FROM comissao
GROUP BY id_comissao;
```

Este exemplo agrupará as comissões pelo `id_comissao` e calculará a soma total dos valores para cada `id_comissao` ¹.

Cláusula HAVING:

Enquanto a cláusula `WHERE` filtra linhas *antes* do agrupamento, a cláusula `HAVING` filtra grupos *após* a aplicação das funções de agregação. Isso permite que você defina condições em resultados agregados ².

Exemplo:

SQL

```
SELECT id_comissao, SUM(valor) AS ValorTotalComissao
FROM comissao
GROUP BY id_comissao
HAVING SUM(valor) > 1000;
```

Este exemplo retornaria apenas os grupos de `id_comissao` onde a soma total das comissões é maior que 1000.

Subconsultas (Consultas Aninhadas)

Subconsultas, também conhecidas como consultas aninhadas ou *nested queries*, são consultas SQL que são incorporadas dentro de outra consulta SQL. Elas podem ser usadas em várias partes de uma instrução SQL, como na cláusula `WHERE`, `FROM` (como tabelas derivadas) ou `SELECT`. O principal objetivo de uma subconsulta é retornar dados que serão usados pela consulta externa ¹.

Exemplo de Subconsulta no `WHERE` :

Para encontrar o nome do cliente que fez o pedido de código 1010:

SQL

```
SELECT clientes.nome
FROM clientes
WHERE clientes.codigo = (
    SELECT pedidos.codcli
    FROM pedidos
    WHERE pedidos.codigo = 1010
);
```

Subconsultas vs. JOINS:

Embora subconsultas possam, em alguns casos, alcançar resultados semelhantes aos `JOIN`s, geralmente os `JOIN`s são mais eficientes em termos de desempenho, especialmente com grandes volumes de dados ³ ⁴ ⁵. Subconsultas podem ser mais

legíveis para certas lógicas complexas, mas `JOIN` s são preferíveis quando a intenção é combinar dados de tabelas relacionadas ⁶ .

Junções (JOINS) de Tabelas

A operação `JOIN` é fundamental em bancos de dados relacionais, permitindo combinar linhas de duas ou mais tabelas com base em uma coluna relacionada entre elas. O objetivo é aglutinar informações distribuídas em diferentes tabelas para formar um conjunto de resultados mais completo e significativo ¹ .

Tipos de JOINS:

1. INNER JOIN

O `INNER JOIN` retorna apenas as linhas que possuem correspondência em *ambas* as tabelas, com base na condição de junção especificada. É o tipo de `JOIN` mais comum e o padrão quando `JOIN` é usado sem um qualificador explícito ¹ .

Sintaxe:

SQL

```
SELECT <campos>
FROM <tabela1>
INNER JOIN <tabela2>
ON <condição_de_junção>;
```

Exemplo:

Para listar todos os pedidos e os dados dos clientes que os fizeram:

SQL

```
SELECT pedidos.*, clientes.*
FROM pedidos
INNER JOIN clientes
ON pedidos.codcli = clientes.codigo;
```

2. OUTER JOINS

Os `OUTER JOIN` s são usados quando se deseja incluir todas as linhas de uma das tabelas (ou de ambas), mesmo que não haja correspondência na outra tabela. As colunas da tabela sem correspondência serão preenchidas com `NULL` ¹ .

a. LEFT (OUTER) JOIN

O `LEFT JOIN` (ou `LEFT OUTER JOIN`) retorna todas as linhas da tabela à esquerda (`FROM`) e as linhas correspondentes da tabela à direita (`JOIN`). Se não houver correspondência na tabela da direita, as colunas da tabela da direita terão valores `NULL` ¹.

Sintaxe:

SQL

```
SELECT <campos>
FROM <tabela_esquerda>
LEFT JOIN <tabela_direita>
ON <condição_de_junção>;
```

Exemplo:

Para listar todos os clientes e seus pedidos, incluindo clientes que não fizeram nenhum pedido:

SQL

```
SELECT clientes.*, pedidos.*
FROM clientes
LEFT JOIN pedidos
ON clientes.codigo = pedidos.codcli;
```

b. RIGHT (OUTER) JOIN

O `RIGHT JOIN` (ou `RIGHT OUTER JOIN`) é o oposto do `LEFT JOIN`. Ele retorna todas as linhas da tabela à direita (`JOIN`) e as linhas correspondentes da tabela à esquerda (`FROM`). Se não houver correspondência na tabela da esquerda, as colunas da tabela da esquerda terão valores `NULL` ¹.

Sintaxe:

SQL

```
SELECT <campos>
FROM <tabela_esquerda>
RIGHT JOIN <tabela_direita>
ON <condição_de_junção>;
```

Exemplo:

Para listar todos os pedidos e os dados dos clientes que os fizeram, incluindo pedidos que podem não ter um cliente correspondente (embora isso seja raro em um banco de dados

bem modelado com chaves estrangeiras):

SQL

```
SELECT clientes.*, pedidos.*
FROM clientes
RIGHT JOIN pedidos
ON clientes.codigo = pedidos.codcli;
```

c. FULL (OUTER) JOIN

O `FULL JOIN` (ou `FULL OUTER JOIN`) retorna todas as linhas quando há uma correspondência em uma das tabelas. É a combinação de `LEFT JOIN` e `RIGHT JOIN`. Ele retorna todas as linhas de ambas as tabelas, preenchendo com `NULL` onde não há correspondência em uma das tabelas ¹.

Sintaxe:

SQL

```
SELECT <campos>
FROM <tabela1>
FULL OUTER JOIN <tabela2>
ON <condição_de_junção>;
```

Exemplo:

Para listar todos os clientes e todos os pedidos, mostrando correspondências quando existem e `NULL` onde não há:

SQL

```
SELECT clientes.*, pedidos.*
FROM clientes
FULL OUTER JOIN pedidos
ON clientes.codigo = pedidos.codcli;
```

3. CROSS JOIN (Produto Cartesiano)

O `CROSS JOIN` retorna o produto cartesiano das duas tabelas, ou seja, combina cada linha da primeira tabela com cada linha da segunda tabela. Isso resulta em um número de linhas igual ao produto do número de linhas de ambas as tabelas. Geralmente é usado com cautela, pois pode gerar conjuntos de resultados muito grandes e raramente é o que se deseja, a menos que se esteja construindo uma tabela de todas as combinações possíveis

¹.

Sintaxe:

SQL

```
SELECT <campos>  
FROM <tabela1>  
CROSS JOIN <tabela2>;
```

Exemplo:

SQL

```
SELECT clientes.nome, clientes.codigo, pedidos.codigo  
FROM clientes  
CROSS JOIN pedidos;
```

Este exemplo geraria uma lista onde cada cliente é combinado com cada pedido, independentemente de haver uma relação lógica entre eles ¹.

Boas Práticas e Considerações Finais

Ao trabalhar com consultas SQL complexas, é importante seguir algumas boas práticas para garantir a legibilidade, manutenção e desempenho:

- **Legibilidade:** Use formatação consistente, recuo e comentários para tornar suas consultas fáceis de entender.
- **Apelidos (Aliases):** Utilize apelidos para nomes de tabelas e colunas, especialmente em JOINs e subconsultas, para evitar ambiguidades e reduzir o tamanho da consulta (ex: `SELECT c.nome FROM clientes c`).
- **Evitar SELECT * :** Selecione apenas as colunas de que você realmente precisa. Isso melhora o desempenho e reduz o tráfego de rede.
- **Otimização:** Entenda como o otimizador de consultas do seu SGBD funciona. Em alguns casos, reescrever uma subconsulta como um JOIN pode melhorar significativamente o desempenho.
- **Índices:** Garanta que as colunas usadas em condições WHERE, JOIN e ORDER BY estejam indexadas adequadamente para acelerar a recuperação de dados.

Dominar consultas SQL complexas é uma habilidade essencial para qualquer profissional que trabalhe com bancos de dados, permitindo a extração de informações valiosas e a construção de sistemas robustos e eficientes.

Referências

[1] Aula 7 - SQL DML (Strutured Query Language – Consultas Complexas – Grupos e Junções. (s.d.). PDF fornecido pelo usuário.

[2] W3Schools. (s.d.). SQL HAVING Clause. Disponível em:

[3] Navicat. (2021, 18 de fevereiro). Joins versus Subqueries: Which Is Faster?. Disponível em:

[4] GeeksforGeeks. (2025, 12 de julho). SQL Join vs Subquery. Disponível em:

[5] Medium. (2024, 22 de fevereiro). SQL Joins vs Subqueries: A Comprehensive Comparison. Disponível em:

[6] Reddit. (2023, 22 de agosto). Join vs SubQuery : r/SQLOptimization. Disponível em: